

Nyx

Sicherheits- und Architekturkonzept

Tor Onion Services + Rust + Dioxus + MLS

Stand: 27. August 2026

Wichtiger Hinweis: Dieses Dokument beschreibt ein Sicherheitsdesign, keinen Beweis absoluter "Abhörsicherheit". Ein produktiver Messenger benötigt Implementierungstests, Dependency-Audits, Fuzzing und eine unabhängige Sicherheitsprüfung.

1. Zielbild

Nyx ist ein Tor-only Messenger, bei dem Transportanonymisierung und Nachrichtenverschlüsselung strikt getrennt werden. Tor schützt den Netzwerkpfad und Onion-Endpunkte; die eigentliche Ende-zu-Ende-Verschlüsselung erfolgt unabhängig davon mit MLS (Messaging Layer Security, RFC 9420) über OpenMLS.

- Kein Clearnet-Fallback: Ohne Tor keine Netzwerkkommunikation.
- Keine Telefonnummer, E-Mail-Adresse oder zentrale Benutzerregistrierung erforderlich.
- Mailbox-Server speichern ausschließlich opaque Zustellinformationen und Ciphertext.
- Optional direkte Onion-Verbindung zwischen Peers, wenn beide gleichzeitig online sind.
- Lokale Schlüssel und Chatdaten werden verschlüsselt gespeichert; Transport- und Identitätsschlüssel bleiben getrennt.

2. Sicherheitsprinzipien

Defense in Depth: Tor allein reicht nicht. Netzwerk-Anonymität, E2EE, lokale Datensicherheit und Metadatenminimierung sind voneinander unabhängige Schutzschichten.

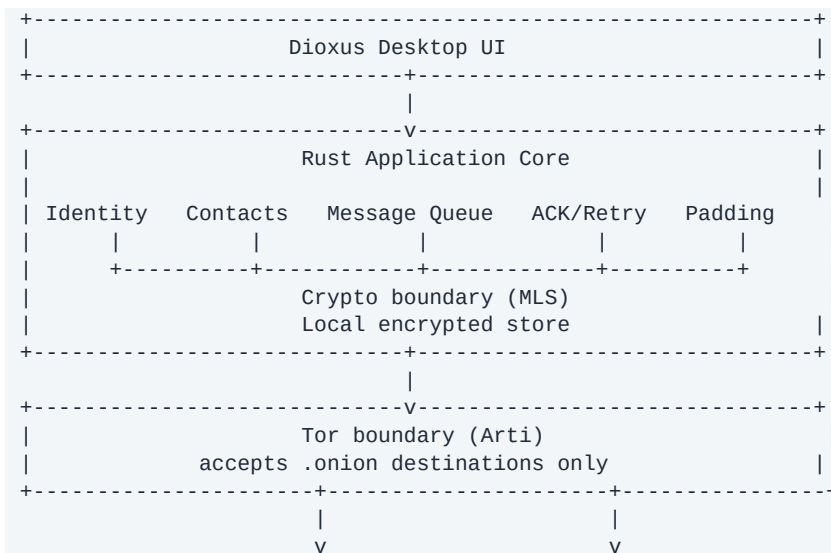
Fail Closed: Tor-Bootstrap oder Onion-Auflösung fehlgeschlagen bedeutet: Nachricht nicht senden. Niemals automatisch direkt verbinden.

Keine Eigenbau-Kryptographie: Session- und Gruppenschlüssel werden über einen standardisierten, etablierten Mechanismus verwaltet. Keine selbst entworfenen Ratchets oder KDF-Konstruktionen.

Minimale Metadaten: Server kennen weder Klartext noch Kontaktlisten, globale Benutzernamen oder langfristige Empfänger-IDs.

Auditierbare Grenzen: UI, Tor, E2EE, Wire-Protokoll und Storage liegen in getrennten Rust-Crates.

3. Gesamtarchitektur



4. Vertrauensmodell

Der Mailbox-Server wird als potenziell kompromittiert behandelt. Daraus folgt: Der Server darf kein Geheimnis besitzen, das zum Entschlüsseln einer Nachricht notwendig ist. Seine Rolle beschränkt sich auf das kurzzeitige Speichern und Ausliefern verschlüsselter Blobs.

Komponente	Darf wissen	Darf nicht wissen
Client	eigene Identität, Kontakte, Klartext nach lokaler Entschlüsselung	fremde Schlüssel/Chats außerhalb eigener Gruppen
Tor-Netz	notwendige Relay-Information pro Hop	vollständigen Kommunikationspfad an einem einzelnen Relay
Mailbox	Mailbox-Token, Ciphertext, Größe, Zeit, TTL	Klartext, Kontaktliste, dauerhafte Benutzeridentität
Peer	Klartext der eigenen Unterhaltung	lokale Secrets des Gegenübers

5. Identität und Kontaktverifikation

Die dauerhafte Benutzeridentität ist ein kryptographischer Identity Key und ausdrücklich nicht die Onion-Adresse. Onion-Service-Schlüssel dürfen rotiert werden, ohne dass dadurch eine neue Benutzeridentität entsteht.

```
Identity Root Key
|
+-- Device Key: Desktop
+-- Device Key: Notebook
+-- Device Key: Mobile (spaeter)
|
+-- signiert aktuellen Onion-Endpunkt
+-- signiert Key Packages / Device-Bindings
```

Kontaktaufnahme erfolgt out-of-band über QR-Code oder Fingerprint. Für sensible Kommunikation muss die Anwendung eine explizite Verifikation anzeigen und Änderungen an Identity-/Device-Keys deutlich melden.

6. Ende-zu-Ende-Verschlüsselung

Als E2EE-Schicht ist MLS nach RFC 9420 vorgesehen. MLS wurde für asynchrone Gruppenkommunikation entwickelt und bietet Forward Secrecy sowie Post-Compromise Security. OpenMLS dient als Rust-Implementierung. Tor ersetzt diese Schicht nicht.

- 1:1-Chats werden als kleine MLS-Gruppe behandelt.
- Gruppenchats verwenden dieselbe Sicherheitsarchitektur und Schlüsselrotation.
- Key Packages und Device-Zustände werden lokal verwaltet.
- Kryptographische Debug-Features dürfen in Release-Builds nicht aktiviert sein.

- Vor Produktionsbetrieb: genaue Ciphersuite, Credential-Modell und Persistenzmodell festlegen und testen.

7. Zustellung und Offline-Modell

Ein rein direktes Peer-to-Peer-Modell ist für einen Messenger unpraktisch, weil Nachrichten bei offline befindlichem Empfänger nicht zugestellt werden können. Deshalb verwendet der Standardmodus einen minimalen Onion-Mailbox-Service.

Alice	mailboxxxxxxxx.onion	Bob
-- Tor + encrypted envelope ----->		
	<---- Tor poll mailbox token --	
	----- encrypted envelope ----->	
	<----- opaque ACK -----	
	----- delete / expire ----->	

Der Server sollte nur mailbox_token, message_id bzw. opaque object id, ciphertext und TTL verarbeiten. Keine Accounts, Passwörter, Kontaktlisten oder Klartext-Metadaten.

8. Nachrichtenformat und Padding

Das äußere Protokoll bleibt bewusst klein. Inhaltstyp, Antwortbezug, Absendergerät und fachliche Metadaten gehören in den verschlüsselten inneren Datensatz.

```
Envelope {
  version,
  mailbox_token,
  ciphertext
}

EncryptedMessage {
  message_id,
  conversation_id,
  sender_device,
  timestamp,
  content_type,
  content,
  reply_to,
  attachment_descriptor
}
```

Nachrichtenlängen verraten Metadaten. Deshalb sollte ein Bucket-Padding verwendet werden, zum Beispiel 512 B, 1 KiB, 2 KiB, 4 KiB, 8 KiB usw. Ein High-Security-Modus kann Textnachrichten auf feste Framegrößen normalisieren und Polling zeitlich bündeln.

9. Dateianhänge

Dateien werden lokal mit einem zufälligen File Key verschlüsselt, bevor sie einen Client verlassen. Der File Key wird ausschließlich innerhalb der E2EE-Nachricht übertragen. Der Server erhält nur einen verschlüsselten Blob.

```
photo.jpg -> random FileKey -> AEAD encryption -> opaque blob
|
+-- FileKey nur innerhalb MLS-Nachricht
```

10. Lokale Datensicherheit

Eine verschlüsselte lokale Datenbank ist Pflicht. Der Master Key sollte bevorzugt aus dem OS-Keychain/Secret-Store stammen. Ein Passwort-Fallback kann über Argon2id einen Schlüssel ableiten. Secret-Key-Material sollte nach Verwendung soweit praktikabel zeroized werden.

- Keine Klartext-Logs von Nachrichten oder Schlüsseln.
- Crash Dumps und Debug-Ausgaben als potenzielle Secret-Leaks behandeln.
- Backup als separates, stark verschlüsseltes Recovery Bundle.
- Geräteentzug und Key-Rotation als eigenes Protokollereignis vorsehen.

11. Bedrohungsmodell

Bedrohung	Schutzwirkung	Grenze
Lokales/ISP-Mitschnueffeln	Tor verschleiern Ziel und Kommunikationspfad; E2EE schützt Inhalt.	Tor-Nutzung selbst kann erkennbar sein.
Kompromittierte Mailbox	Nur Ciphertext und minimale Zustellmetadaten.	Timing, Größe und Zugriffsmuster bleiben beobachtbar.
Einzelnes Tor-Relay	Kein einzelner Relay-Hop kennt den gesamten Pfad.	Globale Korrelation ist nicht vollständig lösbar.
Gestohlene alte Session-Daten	MLS zielt auf Forward Secrecy.	Konkrete Sicherheit hängt von korrekter State-Persistenz/Updates ab.
Kompromittierter Client	Keine zuverlässige Schutzwirkung gegen Keylogger/Screen Capture.	Endpoint Security bleibt außerhalb des Protokolls kritisch.
Bösartiger Gesprächspartner	Keine technische Verhinderung des Kopierens von Klartext nach Empfang.	Empfänger ist legitimer Endpunkt.

12. High-Security-Modus

- Rotierende pseudonyme Mailbox-Tokens pro Zeit-Epoche.
- Feste oder gebucketete Framegrößen.
- Keine Presence-, Typing- oder standardmäßigen Read-Receipt-Signale.
- Batched Polling und optional kontrollierter Cover Traffic.
- Kurze TTL für nicht zugestellte Nachrichten.
- Private Onion Services / Client Authorization für spezielle Deployments.
- Mehrere unabhängige Mailbox-Provider, um zentrale Beobachtung zu erschweren.

13. Rust- und Dioxus-Projektstruktur

```
nyx/
+-- Cargo.toml
+-- README.md
+-- SECURITY.md
+-- THREAT_MODEL.md
```

```

+-- docs/
|   +-- architecture.md
+-- crates/
|   +-- nyx-core/
|   +-- nyx-crypto/
|   +-- nyx-protocol/
|   +-- nyx-store/
|   +-- nyx-tor/
|   +-- nyx-ui/
+-- apps/
    +-- desktop/
    +-- mailbox-server/

```

Die Trennung ist sicherheitsrelevant: Dioxus darf nicht direkt eigene Netzwerkverbindungen aufbauen; alle externen Verbindungen laufen über nyx-tor. Kryptographie ist ausschließlich über nyx-crypto erreichbar. Das Wire-Format kennt keine UI-Strukturen.

14. Technologie-Stack (Projektstand)

Bereich	Vorgesehene Technik	Bemerkung
Sprache	Rust	Memory Safety, klare Crate-Grenzen
Desktop UI	Dioxus 0.7.10	aktueller stabiler 0.7.x Stand; 0.8 war zum Dokumentstand Alpha
Async Runtime	Tokio	für Netzwerk und Tasks
Tor	arti-client 0.45.0	Tor-Client-API; Onion-Service-Features explizit aktivieren
E2EE	OpenMLS 0.8.1	MLS nach RFC 9420
Serialisierung	postcard / alternativ CBOR	kompakt, binär
Lokaler Store	SQLite + zusätzliche Verschlüsselung	Key nicht in DB selbst speichern
KDF	Argon2id	nur für passwortbasierten Fallback
Datei-AEAD	ChaCha20-Poly1305	File Keys innerhalb E2EE verteilen

15. Entwicklungsphasen

Phase 1 - UI/Domain: Dioxus Desktop, Contact/Conversation Models, lokale Testdaten, keine Netzkommunikation.

Phase 2 - Tor-only Transport: Arti Bootstrap, .onion-only Validation, Test-Onion-Service, harte Fail-Closed-Tests.

Phase 3 - MLS 1:1: Identity/Device Credentials, Key Packages, Session State, Encrypt/Decrypt, Persistenztests.

Phase 4 - Mailbox: PUT/GET/ACK/TTL, Limits, Abuse-Schutz ohne Benutzeraccounts, keinerlei Klartext-Logs.

Phase 5 - Metadata Hardening: Padding, rotierende Tokens, Polling-Strategie, Receipt-Policy.

Phase 6 - Gruppen/Multi-Device: MLS-Gruppen, Geräteaufnahme/-entzug, Recovery/Backup.

Phase 7 - Audit/Hardening: Fuzzing, cargo audit, Threat-Model-Review, externer Security Audit.

16. Sicherheits-Abnahmekriterien für den MVP

- Bei blockiertem Tor darf keine direkte Socket-Verbindung zu Mailbox oder Peer entstehen.
- Ein Dump der Mailbox-Datenbank darf keinen Klartext und keine Kontaktliste enthalten.
- Falscher oder geänderter Identity Key löst eine sichtbare Verifikationswarnung aus.
- Malformed network frames dürfen keinen Panic verursachen.
- Message Parser erhält Fuzz-Tests und harte Größenlimits.
- Sicherheitskritische Dependencies sind gepinnt/gelockt und auditiert.
- Logs sind auf Secret-Leaks geprüft; sensible Felder werden nie unredacted ausgegeben.
- Die lokale DB bleibt ohne OS-/Recovery-Key unlesbar.

17. Referenzen

RFC 9420 - The Messaging Layer Security (MLS) Protocol

<https://www.rfc-editor.org/rfc/rfc9420.html>

OpenMLS - Rust implementation of MLS

<https://github.com/openmls/openmls>

Dioxus 0.7 documentation

<https://dioxuslabs.com/learn/0.7/>

arti-client documentation

https://docs.rs/arti-client/0.45.0/arti_client/

Tor Project - Onion Services overview

<https://community.torproject.org/onion-services/overview/>

18. Ergebnis

Das vorgeschlagene Design ist technisch realistisch und vermeidet den häufigsten Architekturfehler: Tor mit E2EE gleichzusetzen. Die entscheidenden Schutzgrenzen sind Tor-only Transport, standardisierte E2EE, minimale Serverkenntnis und eine sichere lokale Schlüsselverwaltung. Das mitgelieferte Cargo-Workspace-Gerüst bildet genau diese Grenzen ab, implementiert die sicherheitskritischen Tor- und MLS-Pfade aber absichtlich noch nicht als scheinbar fertige Lösung.